Week 1 Lab 2 — Pandas Dataframe Refresher

28th October 2026

1 DataFrames

Throughout the course, most of our data exploration will be done with DataFrames. DataFrames are an extremely powerful tool and a natural extension of the Pandas Series. By definition all a DataFrame is:

A Pandas DataFrame consists of multiple Pandas Series that share index values.

[1]:
```
import numpy as np
import pandas as pd
np.random.seed(101)
data = np.random.randint(0,50,(5,4))
```

[2]:
```
data
```

[2]:
```
array([[31, 11, 17,  6],
       [23, 11, 47,  9],
       [13, 40,  4, 40],
       [28,  0, 46,  5],
       [12, 29, 40, 49]])
```

[3]:
```
type(data)
```

[3]:
```
numpy.ndarray
```

[4]:
```
rows = ['SYD', 'MEL', 'CAN', 'BRI', 'PER']
```

[5]:
```
columns = ['JAN', 'FEB', 'MAR', 'APR']
```

[6]:
```
df = pd.DataFrame(data=data)
display(df)
```

```
    0   1   2   3
0  31  11  17   6
1  23  11  47   9
2  13  40   4  40
3  28   0  46   5
4  12  29  40  49
```

[7]:
```
df = pd.DataFrame(data=data,index=rows)
display(df)
```

	0	1	2	3
SYD	31	11	17	6
MEL	23	11	47	9
CAN	13	40	4	40
BRI	28	0	46	5
PER	12	29	40	49

[8]:
```
df = pd.DataFrame(data=data,columns=columns)
display(df)
```

	JAN	FEB	MAR	APR
0	31	11	17	6
1	23	11	47	9
2	13	40	4	40
3	28	0	46	5
4	12	29	40	49

[9]:
```
df = pd.DataFrame(data=data,index=rows,columns=columns)
display(df)
```

	JAN	FEB	MAR	APR
SYD	31	11	17	6
MEL	23	11	47	9
CAN	13	40	4	40
BRI	28	0	46	5
PER	12	29	40	49

You have two options when reading a file with pandas:

1. The code may look like this : df = pd.read_csv('datafile.csv') - this is when your csv file you want to import is in the same folder location of your notebook.
2. The code may look like this : df = pd.read_csv("C:\Projects\ML\files\datafile.csv")v- this is to pass the whole file path if the file is in a different directory

Context

The Matildas - the Australian women's national football team - have played 161 full internationals since the start of 2015. Each row of this dataset is one match: who we played, the year, goals scored and conceded, the crowd, whether it was played in Australia, friendly or tournament, and the result.

Acknowledgements

Match results compiled from the Wikipedia match record, cross-checked against the open women's international results dataset by Mart Jürisoo. Attendance is missing for some older away matches - it was never recorded. Compiled August 2026.

Additional notes

From the Matildas' perspective:

- **Goals scored**: goals the Matildas scored against their opponents.
- **Goals conceded**: goals their opponents scored against them.

[10]:
```
df = pd.read_csv('data/matildas.csv')
```

[11]:
```
#display(df)
df
```

[11]:
```
        opponent  year  goals_scored  goals_conceded  attendance in_australia  \
0    North Korea  2015             2               1         NaN            No
1    New Zealand  2015             3               2         NaN            No
2    Netherlands  2015             1               0         NaN            No
3        England  2015             0               3         NaN            No
4        Finland  2015             3               0         NaN            No
..           ...   ...           ...             ...         ...           ...
156        Japan  2026             0               1     74397.0           Yes
157       Malawi  2026             5               0         NaN            No
158        Kenya  2026             2               0         NaN            No
159       Mexico  2026             0               1     23167.0           Yes
160       Mexico  2026             3               1     18441.0           Yes

     match_type result
0      Friendly      W
1      Friendly      W
2    Tournament      W
3    Tournament      L
4    Tournament      W
..          ...    ...
156  Tournament      L
157  Tournament      W
158  Tournament      W
159    Friendly      L
160    Friendly      W

[161 rows x 8 columns]
```

[12]:
```
display(df)
```

```
        opponent  year  goals_scored  goals_conceded  attendance in_australia  \
0    North Korea  2015             2               1         NaN            No
1    New Zealand  2015             3               2         NaN            No
2    Netherlands  2015             1               0         NaN            No
3        England  2015             0               3         NaN            No
4        Finland  2015             3               0         NaN            No
..           ...   ...           ...             ...         ...           ...
156        Japan  2026             0               1     74397.0           Yes
157       Malawi  2026             5               0         NaN            No
```

```
158        Kenya  2026             2               0         NaN          No
159       Mexico  2026             0               1     23167.0         Yes
160       Mexico  2026             3               1     18441.0         Yes

     match_type result
0      Friendly      W
1      Friendly      W
2    Tournament      W
3    Tournament      L
4    Tournament      W
..          …    …
156  Tournament      L
157  Tournament      W
158  Tournament      W
159    Friendly      L
160    Friendly      W

[161 rows x 8 columns]
```

[13]:
```
df.columns
```

[13]:
```
Index(['opponent', 'year', 'goals_scored', 'goals_conceded', 'attendance',
       'in_australia', 'match_type', 'result'],
      dtype='object')
```

[14]:
```
df.index
```

[14]:
```
RangeIndex(start=0, stop=161, step=1)
```

[15]:
```
df.head(5) #Read the first 5 records
```

[15]:
```
       opponent  year  goals_scored  goals_conceded  attendance in_australia  \
0  North Korea  2015             2               1         NaN           No
1  New Zealand  2015             3               2         NaN           No
2  Netherlands  2015             1               0         NaN           No
3      England  2015             0               3         NaN           No
4      Finland  2015             3               0         NaN           No

   match_type result
0    Friendly      W
1    Friendly      W
2  Tournament      W
3  Tournament      L
4  Tournament      W
```

[16]:
```
df.tail(5) #Read the last five records
```

[16]:
```
     opponent  year  goals_scored  goals_conceded  attendance in_australia  \
156     Japan  2026             0               1     74397.0          Yes
157    Malawi  2026             5               0         NaN           No
158     Kenya  2026             2               0         NaN           No
159    Mexico  2026             0               1     23167.0          Yes
160    Mexico  2026             3               1     18441.0          Yes

     match_type result
156  Tournament      L
157  Tournament      W
158  Tournament      W
159    Friendly      L
160    Friendly      W
```

[17]:
```
df.info() #Information on the dataframe
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 161 entries, 0 to 160
Data columns (total 8 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   opponent        161 non-null    object
 1   year            161 non-null    int64
 2   goals_scored    161 non-null    int64
 3   goals_conceded  161 non-null    int64
 4   attendance      101 non-null    float64
 5   in_australia    161 non-null    object
 6   match_type      161 non-null    object
 7   result          161 non-null    object
dtypes: float64(1), int64(3), object(4)
memory usage: 10.2+ KB
```

[18]:
```
len(df) #length or number of records in the data frame
```

[18]:
```
161
```

[19]:
```
df.describe() #describe the dataframe on some commonly used statistical␣
↪parameters
```

[19]:
```
              year  goals_scored  goals_conceded    attendance
count   161.000000    161.000000      161.000000    101.000000
mean   2020.453416      2.273292        1.192547  21919.128713
std       3.517724      2.392665        1.325309  19822.160297
min    2015.000000      0.000000        0.000000      0.000000
25%    2017.000000      1.000000        0.000000   8678.000000
50%    2021.000000      2.000000        1.000000  15748.000000
75%    2023.000000      3.000000        2.000000  28019.000000
max    2026.000000     18.000000        7.000000  76798.000000
```

[20]:
```
df.describe().transpose() #Tranpose the description of the dataframe - row␣
↪become columns
#columns become rows
```

[20]:
```
                count          mean           std     min     25%      50%  \
year            161.0   2020.453416      3.517724  2015.0  2017.0   2021.0
goals_scored    161.0      2.273292      2.392665     0.0     1.0      2.0
goals_conceded  161.0      1.192547      1.325309     0.0     0.0      1.0
attendance      101.0  21919.128713  19822.160297     0.0  8678.0  15748.0

                    75%      max
year             2023.0   2026.0
goals_scored        3.0     18.0
goals_conceded      2.0      7.0
attendance      28019.0  76798.0
```

Some common selection and Indexing functions with Data frame

Read the data from a single column

[21]:
```
df['attendance']
```

[21]:
```
0          NaN
1          NaN
2          NaN
3          NaN
4          NaN
         …
156    74397.0
157        NaN
158        NaN
159    23167.0
160    18441.0
Name: attendance, Length: 161, dtype: float64
```

[22]:
```
type(df['attendance'])
```

[22]:
```
pandas.core.series.Series
```

[23]:
```
# Its a python list of column names, that's why needs double brackets.
df[['attendance','goals_scored']]
```

[23]:
```
     attendance  goals_scored
0           NaN             2
1           NaN             3
2           NaN             1
3           NaN             0
4           NaN             3
..          …             …
```

```
156     74397.0             0
157         NaN             5
158         NaN             2
159     23167.0             0
160     18441.0             3

[161 rows x 2 columns]
```

Creating a new column for the Dataframe

[24]:
```
df['goal_difference'] = df['goals_scored'] - df['goals_conceded']
```

[25]:
```
df.head()
```

[25]:
```
       opponent  year  goals_scored  goals_conceded  attendance in_australia  \
0  North Korea  2015             2               1         NaN           No
1  New Zealand  2015             3               2         NaN           No
2  Netherlands  2015             1               0         NaN           No
3      England  2015             0               3         NaN           No
4      Finland  2015             3               0         NaN           No

   match_type result  goal_difference
0    Friendly      W                1
1    Friendly      W                1
2  Tournament      W                1
3  Tournament      L               -3
4  Tournament      W                3
```

[26]:
```
df['total_goals'] = df['goals_scored'] + df['goals_conceded']
```

[27]:
```
df.head()
```

[27]:
```
       opponent  year  goals_scored  goals_conceded  attendance in_australia  \
0  North Korea  2015             2               1         NaN           No
1  New Zealand  2015             3               2         NaN           No
2  Netherlands  2015             1               0         NaN           No
3      England  2015             0               3         NaN           No
4      Finland  2015             3               0         NaN           No

   match_type result  goal_difference  total_goals
0    Friendly      W                1            3
1    Friendly      W                1            5
2  Tournament      W                1            1
3  Tournament      L               -3            3
4  Tournament      W                3            3
```

Manipulating columns using Numpy

[28]:
```
# Because pandas is based on numpy, we get awesome capabilities with numpy's␣
↪universal functions!
df['attendance_thousands'] = np.round(df['attendance'] / 1000, 2)
```

Removing columns from the data frame

[29]:
```
df = df.drop("goal_difference",axis=1)
```

[30]:
```
df.head()
```

[30]:

	opponent	year	goals_scored	goals_conceded	attendance	in_australia	match_type	result	total_goals	attendance_thousands
0	North Korea	2015	2	1	NaN	No	Friendly	W	3	NaN
1	New Zealand	2015	3	2	NaN	No	Friendly	W	5	NaN
2	Netherlands	2015	1	0	NaN	No	Tournament	W	1	NaN
3	England	2015	0	3	NaN	No	Tournament	L	3	NaN
4	Finland	2015	3	0	NaN	No	Tournament	W	3	NaN

[31]:
```
df.index
```

[31]:
```
RangeIndex(start=0, stop=161, step=1)
```

[32]:
```
df.set_index('opponent')
```

[32]:

opponent	year	goals_scored	goals_conceded	attendance	in_australia	match_type	result	total_goals	attendance_thousands
North Korea	2015	2	1	NaN	No	Friendly	W	3	NaN
New Zealand	2015	3	2	NaN	No	Friendly	W	5	NaN
Netherlands	2015	1	0	NaN	No				
England	2015	0	3	NaN	No				
Finland	2015	3	0	NaN	No				
…	…	…	…	…	…				
Japan	2026	0	1	74397.0	Yes				
Malawi	2026	5	0	NaN	No				
Kenya	2026	2	0	NaN	No				
Mexico	2026	0	1	23167.0	Yes				
Mexico	2026	3	1	18441.0	Yes				

```
Netherlands  Tournament      W            1                    NaN
England      Tournament      L            3                    NaN
Finland      Tournament      W            3                    NaN
…                 …    …          …                    …
Japan        Tournament      L            1                  74.40
Malawi       Tournament      W            5                    NaN
Kenya        Tournament      W            2                    NaN
Mexico         Friendly      L            1                  23.17
Mexico         Friendly      W            4                  18.44

[161 rows x 9 columns]
```

[33]:
```
df = df.reset_index()
```

[34]:
```
df.head()
```

[34]:
```
   index     opponent  year  goals_scored  goals_conceded  attendance  \
0      0  North Korea  2015             2               1         NaN
1      1  New Zealand  2015             3               2         NaN
2      2  Netherlands  2015             1               0         NaN
3      3      England  2015             0               3         NaN
4      4      Finland  2015             3               0         NaN

  in_australia  match_type result  total_goals  attendance_thousands
0           No    Friendly      W            3                   NaN
1           No    Friendly      W            5                   NaN
2           No  Tournament      W            1                   NaN
3           No  Tournament      L            3                   NaN
4           No  Tournament      W            3                   NaN
```

Work with Rows

[35]:
```
df.iloc[0]
```

[35]:
```
index                          0
opponent                North Korea
year                          2015
goals_scored                     2
goals_conceded                   1
attendance                     NaN
in_australia                    No
match_type                Friendly
result                           W
total_goals                      3
attendance_thousands           NaN
Name: 0, dtype: object
```

[36]:
```
df.loc[2]
```

[36]:
```
index                          2
opponent                Netherlands
year                       2015
goals_scored                  1
goals_conceded                0
attendance                  NaN
in_australia                 No
match_type             Tournament
result                        W
total_goals                   1
attendance_thousands        NaN
Name: 2, dtype: object
```

[37]:
```
df.iloc[0:4]
```

[37]:

	index	opponent	year	goals_scored	goals_conceded	attendance	in_australia	match_type	result	total_goals	attendance_thousands
0	0	North Korea	2015	2	1	NaN	No	Friendly	W	3	NaN
1	1	New Zealand	2015	3	2	NaN	No	Friendly	W	5	NaN
2	2	Netherlands	2015	1	0	NaN	No	Tournament	W	1	NaN
3	3	England	2015	0	3	NaN	No	Tournament	L	3	NaN

1.1 Worked example: `loc` vs `iloc`

Both select rows and columns using square brackets: `example.loc[rows, columns]` or `example.iloc[rows, columns]`. Omitting the column selector selects all columns.

Feature	`loc`	`iloc`
Selects using	Index and column **labels**	Integer **positions**, starting at 0
Single value	`example.loc['SYD', 'FEB']`	`example.iloc[0, 1]`
Slice endpoint	Ending label is **included**	Ending position is **excluded**
Meaning of an integer	A label, if the index contains integers	A position

The small, fictional dataset below uses the city and month labels from earlier in this notebook. It is self-contained and uses `indexing_example` so the existing `df` is preserved.

[38]:
```
import pandas as pd
from IPython.display import display

indexing_example = pd.DataFrame(
```

```
    {'JAN': [10, 20, 30, 40],
     'FEB': [11, 21, 31, 41],
     'MAR': [12, 22, 32, 42]},
    index=['SYD', 'MEL', 'CAN', 'BRI']
)
display(indexing_example)
```

```
     JAN  FEB  MAR
SYD   10   11   12
MEL   20   21   22
CAN   30   31   32
BRI   40   41   42
```

1.1.1 1. Select a row or a single value

`SYD` is the label of the first row, at position `0`. `FEB` is the label of the second column, at position `1`. Each pair below selects the same data.

For this unique index, `loc['SYD']` and `iloc[0]` return a **Series**. Passing a list instead — `loc[['SYD']]` and `iloc[[0]]` — keeps the result as a **one-row DataFrame**, which displays as a table in PyCharm. The extra brackets do not select extra data; they preserve the two-dimensional structure. To view the Series as plain text, use `print(indexing_example.loc['SYD'])`.

Selecting one row and one column returns a scalar value.

[39]:
```
print('Row selected by label (one-row DataFrame):')
display(indexing_example.loc[['SYD']])
print('Same row selected by position (one-row DataFrame):')
display(indexing_example.iloc[[0]])

print('SYD / FEB using loc:', indexing_example.loc['SYD', 'FEB'])
print('SYD / FEB using iloc:', indexing_example.iloc[0, 1])  # Both return 11
```

```
Row selected by label (one-row DataFrame):
```

```
     JAN  FEB  MAR
SYD   10   11   12
```

```
Same row selected by position (one-row DataFrame):
```

```
     JAN  FEB  MAR
SYD   10   11   12
```

```
SYD / FEB using loc: 11
SYD / FEB using iloc: 11
```

Note: for some reason I can't use `display` with the single-row Series.

[40]:
```
indexing_example.loc['SYD']
```

[40]:
```
JAN    10
FEB    11
```

```
MAR    12
Name: SYD, dtype: int64
```

[41]:
```
type(indexing_example.loc['SYD'])
```

[41]:
```
pandas.core.series.Series
```

[42]:
```
indexing_example.iloc[0]
```

[42]:
```
JAN    10
FEB    11
MAR    12
Name: SYD, dtype: int64
```

[43]:
```
type(indexing_example.iloc[0])
```

[43]:
```
pandas.core.series.Series
```

1.1.2 2. Select a range of rows and columns

With `loc`, `'SYD':'CAN'` includes `SYD`, `MEL`, and `CAN`; `'JAN':'FEB'` includes both columns. With `iloc`, `0:3` selects row positions 0, 1, and 2, and `0:2` selects column positions 0 and 1. The two results below are identical.

Label slices follow the existing index order; they do not sort city names alphabetically.

[44]:
```
print('loc: includes the ending labels CAN and FEB')
display(indexing_example.loc['SYD':'CAN', 'JAN':'FEB'])

print('iloc: excludes ending positions 3 and 2')
display(indexing_example.iloc[0:3, 0:2])
```

```
loc: includes the ending labels CAN and FEB
```

	JAN	FEB
SYD	10	11
MEL	20	21
CAN	30	31

```
iloc: excludes ending positions 3 and 2
```

	JAN	FEB
SYD	10	11
MEL	20	21
CAN	30	31

1.1.3 3. Integer labels are not necessarily positions

Here we give a copy integer row labels `[10, 20, 0, 30]`. The row **labelled** `0` is now the third row, while the row at **position** `0` is still the first row. Consequently, `loc[0]` and `iloc[0]` return

different rows. Below we use `loc[[0]]` and `iloc[[0]]` to keep those results as one-row DataFrames for table display.

This distinction matters after filtering or reordering a DataFrame: labels can remain unchanged while positions change.

[45]:
```
integer_labels_example = indexing_example.copy()
integer_labels_example.index = [10, 20, 0, 30]
display(integer_labels_example)

print('loc[[0]]: row with label 0 (the third row, JAN = 30)')
display(integer_labels_example.loc[[0]])

print('iloc[[0]]: row at position 0 (the first row, JAN = 10)')
display(integer_labels_example.iloc[[0]])
```

```
    JAN  FEB  MAR
10   10   11   12
20   20   21   22
0    30   31   32
30   40   41   42
```

```
loc[[0]]: row with label 0 (the third row, JAN = 30)
```

```
   JAN  FEB  MAR
0   30   31   32
```

```
iloc[[0]]: row at position 0 (the first row, JAN = 10)
```

```
    JAN  FEB  MAR
10   10   11   12
```

Remember: use `loc` when you know the row/column names, and `iloc` when you know their positions. `loc` includes the last label in a slice; `iloc` stops before the last position.